

Jacob Yankee
COSC 341
Zhang

- I. Reference
 - A. No references were used
- II. Design:
 - A. For inde(), the rev function is used as the instructions do not prevent us from using it there
 - B. For nele(), I could not figure out how to print the desired result using two functions
 - C. For ntri() I could not figure out how to print the desired result using two functions
 - D. For infront1() I could only get it to work with numbers and I am unsure if that is what the professor wants
- III. Difficulty:
 - A. A lot of the restrictions made some of these more annoying than they should be
 - B. I kept running into a match redundant error when doing question 5
 - C. Problems 8 and 9 feel incomprehensible or maybe I'm just not motivated to do these
 - D. There are online resources for how to tackle problem 9 but all of them use built in library functions
 - E. In the end I gave up on 8 and 9 because I simply don't know and I might be too tired for this.
- IV. Comments:
 - A. I am genuinely baffled that the professor himself stated we would not be going over level five recursion then proceeded to give us a level five recursion problem on the homework and the midterm. Is this on purpose?
 - B. And what's with not allowing us to use built-in library functions or append or any of that? What sort of employer is going to ask you to write some code without using certain aspects of the language that make writing code easier? I thought programming was about being lazy!

Appendix One - Source Code

```
(* Function 1 *)
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: helper *)
(* Description: Helper function for inde *)
fun helper(x, nil, _, result) = result
  | helper(x, y::ys, z, result) =
    if y = x then
```

```

    helper(x, ys, z + 1, z::result)
  else
    helper(x, ys, z + 1, result);
(* Function name: inde *)
(* Description: Returns the index of the occurrence of a given value, starting from 1 *)
fun inde(x, L) =
  let
    val result = helper(x, L, 1, nil)
  in
    rev result
  end;

```

```

inde(1, [1,2,1,1,2,2,1]);

```

```

(* Function 2 *)
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: neleHelp *)
(* Description: Helper function *)
fun neleHelp(_, 0) = nil
  | neleHelp(x, n) = x :: neleHelp(x, n-1);
(* Function name: concat *)
(* Description: Concatenation function *)
fun concat(nil, ys) = ys
  | concat(x::xs, ys) = x :: concat(xs, ys);
(* Function name: nele *)
(* Description: Repeats each element in a list n times *)
fun nele(nil, _) = nil
  | nele(x::xs, n) = concat(neleHelp(x, n), nele(xs, n));

```

```

nele([1,2],3);

```

```

(* Function 3 *)
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: factHelp *)
(* Description: helper function for isfact *)
fun factHelp(num, divisor, acc, n) =
  if acc = n then true
  else if acc > n then false

```

```
    else factHelp(num+1, divisor*(num+1), acc*(num+1), n);
(* Function name: isfact *)
(* Description: Determines if a positive integer is a factorial number *)
fun isfact n = factHelp(1, 1, 1, n);
```

```
isfact(120);
```

```
(* Function 4 *)
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: tri *)
(* Description: Returns a triangular number *)
fun tri(0) = 0
  | tri(x) = x + tri(x-1);
(* Function name: ntriHelp *)
(* Description: Creates a list in ascending order *)
fun ntriHelp(0, L) = L
  | ntriHelp(n, acc) = ntriHelp(n - 1, tri(n) :: acc);
(* Function name: ntri *)
(* Description: Generates a list of n triangular numbers from 1 *)
fun ntri(n) = ntriHelp(n, nil);
```

```
ntri(7);
```

```
(* Function 5 *)
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: rmnil *)
(* Description: Removes all nil elements in a list list *)
fun rmnil(nil) = nil
  | rmnil(nil::tl) = rmnil tl
  | rmnil(xs::tl) = xs :: rmnil tl;
```

```
rmnil([[1,2], nil, [3], nil, [4,5], nil]);
```

```
(* Function 6 *)
```

```
(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
```

```

(* Homework 2 *)
(* Function name: smap *)
(* Description: Maps values *)
fun smap(_, nil) = nil
  | smap(F, x::xs) = F(x) ::smap(F, xs);
(* Function name: chCase *)
(* Description: Converts input string to uppercase *)
fun chcaseHelp(L) = smap(fn x => if ord(x) >= 97 andalso ord(x) <= 122 then chr(ord(x)-32) else
chr(ord (x)), L);
fun chCase(s) = implode(chcaseHelp(explode s));

chCase("ChCase");

```

```

(* Function 7 *)

```

```

(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: smap *)
(* Description: Maps values *)
fun smap(_, nil) = nil
  | smap(F, x::xs) = F(x) ::smap(F, xs);
(* Function name: infront1 *)
(* Description: Inserts an element as the head of each element of a list *)
fun infront1(x, L) = smap(fn y => x::y, L);

infront1(1, [[1,2],nil,[3]]);

```

```

(* Function 8 *)
(* I could not figure this out. *)

```

```

(* Function 9 *)
(* I could not figure this out. *)
(* Function 10 *)

```

```

(* Jacob Yankee *)
(* COSC 341 *)
(* Zhang *)
(* Homework 2 *)
(* Function name: filter *)
(* Description: Filters a list *)
fun filter(_, nil) = nil
  | filter(P, x::xs) =

```

```
if P(x) then x::filter(P, xs)
else filter(P, xs);
```

```
(* Function name: range *)
(* Description: Finds the range between two numbers *)
fun range(0, 0) = nil
  | range(x, y) = if x > y then nil else x :: range(x+1, y);
```

```
(* Function name: plist *)
(* Description: Collects all prime numbers up to the given number *)
fun plist(n) =
  let
    fun isPrime(x) =
      let
        fun check(_, 1) = true
          | check(y, z) =
              if x mod y = 0 then false
              else check(y + 1, z - 1);
      in
        check(2, x div 2)
      end;
  in
    filter(isPrime, range(2, n))
  end;

plist(20);
```

Appendix Two - Sample Runs

Function 1:

Inputs:

```
inde(1, [1,2,1,1,2,2,1]);
inde(2,[1,2,1,1,2,2,1]);
```

Outputs:

```
val it = [1, 3, 4, 7]: int list;
val it = [2, 5, 6]: int list;
```

Function 2:

Inputs:

```
nele([1,2],3);
nele([1,2,3],3);
```

Outputs:

```
val it = [1, 1, 1, 2, 2, 2]: int list;
val it = [1, 1, 1, 2, 2, 2, 3, 3, 3]: int list;
```

Function 3:

Inputs:

```
isfact(120);
```

```
isfact(121);
```

Outputs:

```
val it = true: bool;
```

```
val it = false: bool;
```

Function 4:

```
ntri(7);
```

```
ntri(4);
```

Outputs:

```
val it = [1, 3, 6, 10, 15, 21, 28]: int list;
```

```
val it = [1, 3, 6, 10]: int list;
```

Function 5:

Inputs:

```
rmnil([[1,2], nil, [3], nil, [4,5], nil]);
```

```
rmnil([nil, [1, 2], nil]);
```

Outputs:

```
val it = [[1, 2], [3], [4, 5]]: int list list;
```

```
val it = [[1, 2]]: int list list;
```

Function 6:

Inputs:

```
chCase("ChCase");
```

```
chCase("hi there!");
```

Outputs:

```
val it = "CHCASE": string;
```

```
val it = "HI THERE!": string;
```

Function 7:

Inputs:

```
infront1(1, [[1,2],nil,[3]]);
```

```
infront1(3, [nil, [5, 5]]);
```

Outputs:

```
val it = [[1, 1, 2], [1], [1, 3]]: int list list;
```

```
val it = [[3], [3, 5, 5]]: int list list;
```

Function 8:

(* I could not figure this out. *)

Function 9:

(* I could not figure this out. *)

Function 10:

Inputs:

plist(20);

plist(30);

Outputs:

val it = [2, 3, 5, 7, 11, 13, 17, 19]: int list;

val it = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]: int list;